

Minesweeper TDD

Game design implementation

Whole game lives in 5 2d arrays where information is stored about each cell. There are two classes that split the responsibility:

Minesweeper - board operations and game management

Cell - click detection

Technical decisions

1. Grid constructed at runtime from a single prefab - better scalability and maintainability, than placing 100 individual cells or more by hand.
2. 5 arrays instead of a per - cell data class so every operation is a $O(1)$ index with no search for best time complexity.
3. Mine layout happens on first click - prevents first click death, avoiding a poor game design decision.
4. Recursive burst flood - revealing a square with no neighboring mines recursively reveals its eight neighbors. Works well for a small board but could run into stack overflow issue if the board is greatly expanded.

Design patterns

Prototype pattern - Unity's prefab is a configured instance that is copied, which is precisely what Prototype is - new objects are produced by cloning. Spawn loop instantiates one hundred times against one prefab.

Lazy initialization - mine array is not populated until first needed, used as a solution for a game design decision

Naming conventions

Uses standard C#/Unity naming convention